

=== CONTROL ===

- **CLK (Clock)**: Semnalul de ceas al sistemului (100 MHz în testbench). Toate deciziile și tranzițiile de stări se întâmplă pe frontul crescător al acestui semnal.
- **RST (Reset)**: Semnalul de inițializare. Când este activ (1), aduce controllerul în starea **IDLE** și golește/resetează directoarele de tag-uri și stările LRU.

=== CPU === (Interfața dintre Procesor și Cache)

- **cpu_req (CPU Request)**: Un semnal prin care procesorul anunță cache-ul: *"Am nevoie să citesc sau să scriu ceva"*.
- **cpu_rw (CPU Read/Write)**: Indică tipul operației dorite de procesor: **0** înseamnă Citire (Read), **1** înseamnă Scriere (Write).
- **cpu_addr (CPU Address)**: Adresa de 32 de biți din memorie pe care procesorul vrea să o acceseze.
- **cpu_wr_data (CPU Write Data)**: Datele (de 32 biți/1 cuvânt) pe care procesorul vrea să le scrie în cache (folosit doar dacă **cpu_rw** este 1).
- **cpu_rd_data (CPU Read Data)**: Datele pe care cache-ul le returnează către procesor în urma unei citiri reușite.
- **cpu_ready**: Semnalul prin care cache-ul îi spune procesorului: *"Am terminat treaba, datele sunt valide sau scrierea s-a efectuat, poți merge mai departe"*.

=== MEMORIE PRINCIPALA === (Interfața dintre Cache și RAM)

- **mem_req (Memory Request)**: Cache-ul anunță memoria RAM că are nevoie de ea (fie pentru a aduce un bloc la MISS, fie pentru a evacua un bloc la EVICT).
- **mem_rw (Memory Read/Write)**: Direcția transferului cu memoria RAM: **0** = Cache-ul citește un bloc întreg (Fetch), **1** = Cache-ul scrie un bloc în RAM (Evict).
- **mem_addr (Memory Address)**: Adresa de la care se aduce sau la care se scrie blocul de 64 de Octeți în memoria RAM.
- **mem_ready**: Memoria RAM este lentă (are latență de 3 cicluri în TB). Când acest semnal devine **1**, înseamnă că RAM-ul a terminat de citit/scriș blocul cerut de cache.

=== FSM CACHE === (Starea internă a Controllerului)

- **FSM_state**: Starea curentă a automatului de stări (**IDLE**, **TAG_CHECK**, **READ_HIT** etc.). Îți arată exact "unde se află" controllerul în fiecare ciclu de ceas.
- **cache_hit**: Un semnal intern generat în starea **TAG_CHECK**. Devine **1** dacă adresa căutată de procesor se află deja într-unul din cele 4 moduri (ways) valide din cache.
- **victim_way: victim (Calea Victimă)**: Este un indice (între 0 și 3, deoarece cache-ul tău este 4-way set-associative) care indică **care dintre cele 4 blocuri din set a fost ales pentru a fi dat afară**. Alegerea se face pe baza politicii LRU (*Least Recently Used*): este dat afară blocul care n-a mai fost folosit de cel mai mult timp.
- **hit_way**: Dacă avem **cache_hit = 1**, acest field (0, 1, 2 sau 3) îți spune **exact în care dintre cele 4 moduri** (linii din set) au fost găsite datele.

=== TRANZACTII === (Variabile de debug din Testbench)

- **OP 0R-1W**: O copie simplificată a tipului de operație curentă (0 pentru Read, 1 pentru Write) salvată de testbench.
- **HIT-MISS**: Monitorizează dacă tranzacția curentă s-a soldat cu un succes (Hit = 1) sau eșec (Miss = 0).
- **EVICT (Semnal Wave)**: Un puls de scurtă durată generat de modelul de memorie din testbench atunci când recepționează un bloc evacuat.
- **ADDR, WR_DATA, RD_DATA**: Copii ale adreselor și datelor curente, folosite de testbench pentru a le afișa frumos în grafic fără a căuta prin semnalele complexe de magistrală.

=== STATISTICI === (Performanța Cache-ului)

- **TOTAL**: Numărul total de cereri trimise de CPU de la pornirea simulării.
- **HITS**: Câte dintre cereri au fost rezolvate instant din cache (Hit-uri).
- **MISSSES**: Câte cereri au generat un eșec (datele nu erau în cache și a trebuit accesată memoria RAM).
- **EVACUARI: (Evacuări)**: Este un contor statistic. Acesta crește cu +1 de fiecare dată când un bloc modificat (dirty) este trimis înapoi în memoria principală pentru a elibera spațiu.
- **HIT RATE**: Eficiența cache-ului exprimată în procente (între 0% și 100%). Se calculează ca $(HITS / TOTAL) * 100\%$.

=== ADRESA DECODATA === (Cum vede cache-ul adresa CPU)

Adresa de 32 de biți trimisă de CPU este spartă în 3 bucăți pentru a căuta în structura hardware:

- **TAG**: Cei mai semnificativi 18 biți. Este ca un "număr de buletin" al blocului de date. Cache-ul îl folosește pentru a verifica dacă blocul stocat în cache aparține exact adresei cerute.
- **INDEX**: Cei 7 biți din mijloc. Cache-ul tău are 128 de seturi ($2^7 = 128$). Indexul selectează exact rândul (setul) în care se fac căutările.
- **WORD**: Biții care selectează cuvântul de 32 de biți dorit din interiorul blocului de 64 de Octeți (un bloc are 16 cuvinte, deci avem nevoie de 4 biți, deoarece $2^4 = 16$).